

🏆 Java程序设计 期末复习资料v3（真题导向·全知识点覆盖）

本资料以河大软院java的ppt为主体框架，融合3张真题内容，补充常用 API、IO 流等模块，每个知识点均配历年真题或典型例题+详细解析，以 **期末高分** 为导向。

版权声明：本复习文档由 **折木柯南** 制作，内容结合 **梁老师** 课堂讲授重点、课程课件、历年真题综合总结。特别鸣谢：**李玉**和**苦瓜**在整理时提供的帮助，本资料仅供学习使用，严禁售卖

🎉 **考试祝福：**愿你考场上思路清晰，笔下生风，**Java高分**！加油！👏

📋 **题型覆盖：**选择、填空、简答、程序分析、编程题五大题型

🏷️ **标记说明：**★ 高频考点 | ⚠️ 易错点 | 💡 应试技巧 | 📌 本节仅考选填

📊 期末模拟试卷结构（满分 100 分）

题型	题量	分值
一、单项选择题	15 题 × 2 分	30 分
二、填空题	10 题 × 2 分	20 分
三、程序分析题	3 题 × 5 分	15 分
四、简答题	3 题 × 5 分	15 分
五、综合编程题	5 题 × 4 分	20 分
合 计	36 题	100 分

📖 第一部分 分模块考点与真题

模块一 Java 语言基础

1.1 语言基础概念

应试速记（填空/选择高频）

考点	一句话记忆
JVM 作用	运行 .class 字节码，实现跨平台
JRE = ?	JVM + 核心类库
JDK = ?	JRE + 开发工具（javac等）
Java 四大特征	面向对象、跨平台、多线程、健壮安全
Java 语言类型	静态强类型 语言
源文件后缀	.java
编译后文件	.class 字节码

核心知识点

1. JDK、JVM、JRE 是什么？有什么关系？

名称	全称	作用
JVM	Java Virtual Machine	运行 .class 字节码，实现跨平台核心
JRE	Java Runtime Environment	JVM + 核心类库，运行 Java 程序所需环境
JDK	Java Development Kit	JRE + 开发工具（javac、jar 等），开发者使用

关系：JDK ⊃ JRE ⊃ JVM

一句话：用 JDK 开发，用 JRE 运行，JVM 是运行的核心。

2. Java 语言的四大特征

特征	说明
面向对象	封装、继承、多态三大特性

特征	说明
跨平台	一次编译，到处运行（JVM 实现）
多线程	语言内置多线程支持
健壮安全	无指针、自动 GC、异常机制、强类型

补充：Java 是 **静态强类型语言**——所有变量必须先声明类型，编译期进行类型检查（考题常问）

3. 程序编译运行流程

.java 源文件 → [javac 编译] → .class 字节码 → [java 命令运行] → JVM 加载执行

- **编译阶段：**javac 将 .java 编译为 .class 字节码（与平台无关）
- **运行阶段：**java 命令启动 JVM，加载并执行 .class 文件

★ 高频考点提炼

1. JVM 的作用是什么？（跨平台的核心）
2. Java 是不是纯面向对象语言？（是，除了基本数据类型）
3. Java 是编译型还是解释型？（编译 + 解释，先编译为字节码，再由 JVM 解释执行）
4. 四大特征描述（多选/填空题）

做题技巧与避坑指南

- **选择题：**看到"跨平台"选 JVM；看到"开发工具包"选 JDK；看到"运行环境"选 JRE
- **填空题：**Java 源文件后缀是 .java ，编译后后缀是 .class

典型真题演练

题 1（单选）：Java 语言能够实现"一次编译，到处运行"的特性，主要依赖于（ ）
A. JDK B. JRE C. JVM D. 编译器

答案：C
解析：JVM 为不同平台提供对应的实现，.class 字节码在任意平台的 JVM 上都能运行。

题 2（单选）：以下哪个不是 Java 的特征？（ ）
A. 面向对象 B. 多继承 C. 跨平台 D. 多线程

答案：B
解析：Java 类是单继承，接口可以多继承。Java 的特征是面向对象、跨平台、多线程、健壮安全。

题 3（填空）：Java 源程序编译后会生成_____文件，该文件由_____执行。

答案：.class（字节码）；JVM（Java 虚拟机）

题 4（判断）：Java 是纯面向对象的语言，所有的数据类型都是对象。（ ）

答案：×
解析：Java 有 8 种基本数据类型（int、double 等），它们不是对象，但可以通过包装类转为对象。

题 5（单选）：以下关于 JDK、JRE、JVM 的说法正确的是（ ）

- A. JRE 包含了 JDK
- B. JVM 包含了 JRE
- C. JDK 包含了 JRE，JRE 包含了 JVM
- D. 三者相互独立

答案：C
解析：包含关系 $JDK \supset JRE \supset JVM$ 。

题 6（填空）：Java 中负责垃圾回收的机制是_____。

答案：GC（Garbage Collection）

1.2 数据类型

应试速记

类型	字节	默认值	包装类	范围速记
byte	1	0	Byte	-128~127
short	2	0	Short	±3万
int	4	0	Integer	±21亿（默认整数类型）

类型	字节	默认值	包装类	范围速记
long	8	0L	Long	很大，赋值要加L
float	4	0.0f	Float	约±3.4E38，赋值加f
double	8	0.0d	Double	默认小数类型
char	2	'\u0000'	Character	0~65535，Unicode
boolean	1	false	Boolean	true/false

⚠ 考试必记：

- float f = 1.1; ❌ 必须 float f = 1.1f;
- long l = 12345678901; ❌ 必须 long l = 12345678901L;
- byte b = 128; ❌ 超出 -128~127 范围
- 小数默认是 double 类型
- int 是默认整数类型

1.2.2 ★ 字符串（String、StringBuilder、StringBuffer）

String（重点）

- 是引用类型，不是基本数据类型（常考选择题）
- 不可变：String 对象一旦创建，内容不可改变（每次修改都会创建新对象）
- 字符串常量池：JVM 对字符串字面量进行复用（了解原理）

```
String s1 = "hello";           // 常量池
String s2 = "hello";           // 复用常量池对象
String s3 = new String("hello"); // 堆上新对象
System.out.println(s1 == s2);   // true（同一地址）
System.out.println(s1 == s3);   // false（不同地址）
System.out.println(s1.equals(s3)); // true（内容相同）
```

- == 比较地址，equals() 比较内容（重要！）
- 字符串拼接：+ 当两边是字符串时做拼接，不是加法运算
- 常用方法：

方法	说明	示例
length()	返回字符串长度	"abc".length() → 3
charAt(i)	返回第 i 个字符	"abc".charAt(1) → 'b'
substring(i,j)	截取 [i,j) 子串	"hello".substring(1,3) → "el"
indexOf(s)	返回子串首次出现位置	"hello".indexOf("l") → 2
split(regex)	按正则分割	"a,b,c".split(",") → ["a","b","c"]
equals(s)	比较内容是否相等	"abc".equals("abc") → true
toCharArray()	转为 char 数组	"ab".toCharArray() → ['a','b']

StringBuilder 和 StringBuffer

- 可变字符序列，可修改内容，不会创建新对象
- **StringBuffer**：线程安全（方法加了 synchronized），效率较低
- **StringBuilder**：非线程安全，效率高（**推荐单线程使用**）
- 常用方法：append()、insert()、delete()、reverse()、toString()

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");           // "Hello World"
sb.insert(5, ",");             // "Hello, World"
sb.reverse();                  // "dlroW ,olleH"
String result = sb.toString();
```

三者对比：

特性	String	StringBuilder	StringBuffer
可变性	不可变	可变	可变
线程安全	安全（不可变）	不安全	安全
效率	最低（频繁修改）	最高	中
适用场景	少量字符串操作	单线程大量修改	多线程大量修改

1.2.3 变量

1. 局部变量 vs 全局变量（成员变量）

对比项	局部变量	成员变量
定义位置	方法内部/代码块内	类中方法外
默认值	无，必须初始化才能使用	有默认值（0/null/false）
作用域	所在方法/代码块内	整个类
生命周期	方法调用时创建，结束销毁	对象创建时创建，GC 销毁

2. ★ final 关键字

- 修饰 **变量** → 常量，只能赋值一次（声明时必须初始化）
- 修饰 **方法** → 不能被重写
- 修饰 **类** → 不能被继承

```
final int MAX = 100;
// MAX = 200;  // 编译错误！

final class MathUtils { }
// class MyMath extends MathUtils { }  // 编译错误！
```

3. ★ static 关键字

- 修饰的成员属于**类级别**，不属于某个对象
- 通过 **类名.成员名** 直接调用
- **静态方法中不能直接访问非静态成员**（因为静态方法不依赖对象）

```
class Student {
    static String schoolName = "HENU"; // 静态变量
    String name;                       // 实例变量

    static void showSchool() {
        System.out.println(schoolName); // 正确
        // System.out.println(name);    // 编译错误！静态方法不能访问非静态成员
    }
}

// 调用：Student.schoolName; Student.showSchool();
```

★ 高频考点提炼

1. 基本数据类型字节数（填空题常考 int 4、double 8、char 2）
2. String 是引用类型（判断题）
3. == 与 equals() 的区别（必考选择题）
4. final 和 static 关键字的作用（选择题/填空题）
5. StringBuilder 和 StringBuffer 的区别（简答题）
6. 静态方法不能直接访问非静态成员（判断题）

典型真题演练

题 1（单选）：下列哪个是 Java 的基本数据类型？（）

A. String B. boolean C. Integer D. ArrayList

答案：B

解析：A String 是引用类型，C Integer 是 int 的包装类（引用类型），D 是集合类。

题 2（单选）：以下代码输出结果是（）

```
String s = "abcd";  
String s1 = new String(s);  
System.out.println(s == s1);  
System.out.println(s.equals(s1));
```

A. true true B. true false C. false true D. false false

答案：C

解析：s 在常量池，s1 是 new 出来的堆上新对象，地址不同故 == false；内容相同故 equals true。

题 3（单选）：下列哪个关键字用于声明常量？（）

A. static B. final C. const D. abstract

答案：B

解析：final 修饰变量使其成为常量。const 是 Java 保留字但未使用。

题 4（单选）：关于静态方法，以下说法正确的是（）

A. 静态方法可以重写

- B. 静态方法中可以直接访问实例变量
- C. 静态方法属于类，可以通过类名直接调用
- D. 静态方法只能访问静态成员

答案：C

解析：静态方法不能被重写（A错），不能直接访问实例变量（B错），只能直接访问静态成员（D说法太绝对，可以创建对象后访问实例成员）。

题 5（代码输出）：

```
StringBuilder sb = new StringBuilder("Java");  
sb.append("编程");  
System.out.println(sb);
```

答案：Java编程

题 6（简答）：请简述 String、StringBuilder、StringBuffer 三者的区别。

参考答案：

1. **可变性：**String 不可变，每次修改创建新对象；StringBuilder 和 StringBuffer 可变。
2. **线程安全：**String 安全（不可变）；StringBuffer 安全（方法同步）；StringBuilder 不安全。
3. **效率：**String 最低（大量修改时），StringBuilder 最高，StringBuffer 居中。
4. **适用场景：**少量字符串操作用 String；单线程大量修改用 StringBuilder；多线程用 StringBuffer。

题 7（填空）：int 类型占用_____个字节，double 类型占用_____个字节。

答案：4；8

题 8（判断）：Java 中 String 是基本数据类型。（）

答案：X。String 是引用类型。

题 9（代码输出）：

```

public class Test {
    static int a = 10;
    int b = 20;

    public static void main(String[] args) {
        System.out.println(a);
        // System.out.println(b); // 取消注释会怎样?
    }
}

```

答案：输出 10。取消注释会编译错误，因为静态方法 main 不能直接访问非静态成员变量 b。

题 10 (单选)：以下关于 float 变量的定义，正确的是 ()

A. float f = 12.5; B. float f = 12.5f; C. float f = 12.5F; D. B和C都正确

答案：D

解析：float 赋值必须加 f/F 后缀，否则默认 double 无法隐式转为 float。

题 11 (单选)：执行 String s = "Hello"; s = s + " World"; 后，下列描述正确的是 ()

A. 字符串 "Hello" 的内容被修改为 "Hello World"
 B. 创建了一个新的字符串对象 "Hello World"，原 "Hello" 不变
 C. 编译错误
 D. 运行时异常

答案：B

解析：String 不可变，每次拼接都会创建新对象。

题 12 (代码输出)：

```

String str = "hello";
System.out.println(str.length());
System.out.println(str.charAt(1));
System.out.println(str.substring(1, 3));

```

答案：5; e; el

解析：length()=5; charAt(1)='e'; substring(1,3) 取索引[1,3)，即 "el"。

题 13 (填空)：String 类中用于将字符串转换为 char 数组的方法是_____。

答案：toCharArray()

题 14 (程序分析-输出结果)：

```

public class Test {
    public static void main(String[] args) {
        String s1 = "abc";
        String s2 = "abc";
        String s3 = new String("abc");
        System.out.println(s1 == s2);
        System.out.println(s1 == s3);
        System.out.println(s1.equals(s3));
    }
}

```

答案：true; false; true

解析：s1和s2都指向常量池同一对象，故==为true；s3是new出来的堆上新对象，地址不同；equals比较内容相同。

题 15（程序分析-输出结果）：

```

public class Test {
    public static void main(String[] args) {
        int a = 100;
        Integer b = 100;
        Integer c = 100;
        Integer d = 200;
        Integer e = 200;
        System.out.println(a == b);
        System.out.println(b == c);
        System.out.println(d == e);
    }
}

```

答案：true; true; false

解析：ab 基本类型与包装类比较会自动拆箱，比较值；bc 均为100，在缓存池(-128~127)范围内，复用同一对象；d==e 200超出缓存池范围，各自new新对象，地址不同。

1.3 数组

✦ 应试速记

考点	速记
数组是什么类型	引用类型
长度是否可变	不可变（创建后固定）
长度获取	arr.length （属性，无括号）
字符串长度获取	str.length() （方法，有括号）
二维数组行数是否必须指定	必须指定第一维长度

核心知识点

- 1. **Java 数组是引用类型**，长度固定（一旦创建不可改变）
- 2. **声明语法：**

```
int[] arr = new int[5];           // 动态初始化
int[] arr = {1, 2, 3, 4, 5};      // 静态初始化
```
- 3. **数组长度属性：** arr.length （注意不是方法，没有括号！）
- 4. **二维数组：** 每行长度可以不同（不规则数组）

```
int[][] a = new int[3][];           // 合法，只指定行数
int[][] a = new int[3][4];          // 合法，3行4列
// int[][] a = new int[][3];        // 非法！必须指定第一维长度
int[][] a = {{1,2}, {3,4,5}};      // 合法，静态初始化
```

⚠ 易错对比

操作	数组	String
长度获取	arr.length （属性）	str.length() （方法）
是否可变	长度不可变，元素可变	不可变
== 比较	比较地址	比较地址

典型真题演练

题 1 (多选): 下列哪些语句是正确的? ()

- A. `int a[][] = new int[][3];`
- B. `int a[][] = {{1,3}, {2,3,4}, {1,2}};`
- C. `String s[][] = new String[2][];`
- D. `String s[][] = {"can", "I"}, {"help", "you"};`

答案: BCD

解析: 二维数组必须指定第一维长度, 第二维可省略。

题 2 (填空): 获取数组元素个数使用_____属性 (注意不是方法)。

答案: length

题 3 (判断): Java 中数组的长度可以动态改变。()

答案: X。Java 数组一旦创建, 长度固定。

题 4 (单选): 以下数组定义中, 正确的是 ()

- A. `int[] arr = new int[];`
- B. `int[5] arr = new int[5];`
- C. `int[] arr = new int[5];`
- D. `int arr = new int[5];`

答案: C

解析: A 未指定长度; B 声明时不能指定大小; D arr 是数组类型而非 int。

题 5 (代码输出):

```
int[] arr = {1, 2, 3, 4, 5};
System.out.println(arr.length);
```

答案: 5

题 6 (程序分析-输出结果):

```
public class Test {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] b = a;
        b[1] = 100;
        System.out.println(a[1]);
    }
}
```

答案：100

解析：数组是引用类型，b=a 使 b 和 a 指向同一数组对象，修改 b[1] 即修改 a[1]。

题 7（程序分析-输出结果）：

```
public class Test {
    public static void main(String[] args) {
        int[][] arr = new int[2][];
        arr[0] = new int[]{1, 2, 3};
        arr[1] = new int[]{4, 5};
        System.out.println(arr[0].length + "," + arr[1].length);
    }
}
```

答案：3,2

解析：二维数组每行长度可以不同，arr[0]有3个元素，arr[1]有2个元素。

1.4 运算符与表达式

✦ 应试速记

运算符类型	要点
% 取余	5%2 = 1
/ 整数除法	5/2 = 2 （截断，非四舍五入）
&& 短路与	左边false→右边不执行
、	

运算符类型	要点
i++ 后置++	先取值后自增
++i 前置++	先自增后取值
?: 三元	条件?结果1:结果2

核心知识点

- 1. 算术运算符： + - * / %
⚠️ 整数相除截断： 5/2 = 2 （不是 2.5）
- 2. 关系运算符： == != > < >= <=
- 3. 逻辑运算符： && || ! （短路特性）
 - &&： 左边 false 则右边不执行
 - ||： 左边 true 则右边不执行
- 4. 位运算符： & | ^ ~ >> << >>> （了解）
- 5. 三元运算符： 条件 ? 值1 : 值2
- 6. 类型转换：
 - 自动转换：小→大 int → long → float → double
 - 强制转换：大→小 (类型)值

典型真题演练

题 1（代码输出）：

```
int a = 5;
int b = 2;
System.out.println(a / b);
```

答案：2
解析：整数除法结果截断为整数。

题 2（代码输出）：

```
int x = 10;
int y = (x > 5) ? 100 : 200;
System.out.println(y);
```

答案：100

题 3（陷阱题）：

```
int i = 0;
if (i++ == 0 || i++ == 2) {}
System.out.println(i);
```

答案：1

解析：i++ == 0 先判断 (i=0 → true)，然后 i 自增为 1。由于 || 短路，右边不执行。

题 4（代码输出）：

```
int a = 3;
int b = a++ + ++a;
System.out.println(b);
```

答案：8

解析：a++ 先用 a=3 再自增为4，++a 先自增为5再用5，b = 3 + 5 = 8。

题 5（代码输出）：

```
System.out.println(10 % 3);
System.out.println(3 + 4 * 2);
```

答案：1； 11

解析：10%3=1； * 优先级高于 +，先算4*2=8，再3+8=11。

模块二 面向对象核心（考试重点！ 分值占比最高）

★ 本模块是期末考试重中之重，覆盖封装、继承、多态、抽象类、接口五大高频考点，简答题和编程题均大量涉及，请重点复习！

2.1 类与对象

✦ 应试速记

考点	速记
一个 .java 文件几个 public 类	1个
构造方法能否被继承	✗ 不能，只能被调用
无构造方法时	编译器提供默认无参构造
toString() 默认	类名@地址（如 Student@15db9742）
this 作用	区分成员/局部变量、调用本类构造

2.1.1 自定义类

核心语法：

```
public class Student {           // 一个文件只能有一个 public 类
    private String name;         // 属性（成员变量）

    public Student() {}          // 构造方法（与类同名，无返回值）

    public String getName() {     // getter
        return name;
    }

    public void setName(String name) { // setter
        this.name = name;        // this 区分成员变量和参数
    }

    @Override
    public String toString() {    // 重写 toString 避免打印地址
        return "Student{name='" + name + "'}";
    }
}
```

关键规则：

- 1. 一个 .java 文件只能有 1 个 public 类
- 2. 构造方法不能被继承，只能被调用（通过 super()）

- 3. 如果没有定义构造方法，编译器提供**默认无参构造**
- 4. ★ **toString() 方法**：默认打印类名@地址（如 Student@15db9742 ），重写后可打印属性值

2.1.2 ★ 构造方法

特性	说明
方法名	必须与类名相同
返回值	无 （也不能写 void）
调用时机	<code>new</code> 创建对象时自动调用
重载	可以重载（参数列表不同）
继承	不能被继承，子类通过 <code>super()</code> 调用

```
public class Person {
    String name;
    int age;

    // 无参构造（若不写，系统默认提供）
    public Person() {}

    // 有参构造（重载）
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

// 使用
Person p1 = new Person();           // 调用无参
Person p2 = new Person("张三", 20); // 调用有参
```

2.1.3 访问修饰符

修饰符	同一类	同一包	子类（不同包）	所有类
private	✓	X	X	X

修饰符	同一类	同一包	子类（不同包）	所有类
default（不写）	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

权限大小： public > protected > default > private

典型真题演练

题 1（单选）：如果一个类没有显式地声明一个构造方法，那么会（）

- A. 编译错误 B. 运行时错误
C. 提供一个默认的无参构造方法 D. 继承父类的所有构造方法

答案：C

题 2（单选）：以下关于构造方法的描述，错误的是（）

- A. 构造方法名与类名相同
B. 构造方法没有返回值
C. 构造方法可以重载
D. 子类可以继承父类的构造方法

答案：D。构造方法不能被继承，只能被调用。

题 3（填空）：一个 Java 源文件中最多可以定义_____个 public 类。

答案：1

题 4（简答）： this 关键字有哪些用法？

参考答案：

1. 区分成员变量和局部变量（`this.name = name`）
2. 调用本类其他构造方法（`this()`，必须放在构造方法第一行）
3. 返回当前对象引用（`return this;`）

题 5（单选）：以下关于构造方法的说法正确的是（）

- A. 构造方法必须手动定义
B. 构造方法的返回值类型是 void

C. 构造方法可以被重载

D. 构造方法可以被继承

答案：C

解析：A 不定义时编译器会提供默认无参构造；B 构造方法无返回值（也不能写void）；D 构造方法不能被继承。

题 6（程序分析-输出结果）：

```
class Person {
    String name;
    public Person() {
        System.out.print("无参构造 ");
    }
    public Person(String name) {
        this.name = name;
        System.out.print("有参构造 ");
    }
}

public class Test {
    public static void main(String[] args) {
        Person p1 = new Person();
        Person p2 = new Person("张三");
    }
}
```

答案：无参构造 有参构造

解析：new Person() 调用无参构造，new Person("张三") 调用有参构造，体现构造方法重载。

题 7（填空）：在类中，如果没有定义任何构造方法，编译器会自动添加一个_____构造方法。

答案：默认无参

2.2 封装

核心知识点

封装思想：

1. 属性使用 **private** 私有化

2. 提供公共的 **getter/setter** 方法访问
3. 外部无法直接修改内部属性，必须通过方法

封装的好处：

- **安全**：防止非法数据直接赋值
- **解耦**：内部实现变化不影响外部调用
- **复用**：提高代码可维护性

```
public class BankAccount {  
    private double balance; // 私有属性  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void deposit(double money) {  
        if (money > 0) {  
            balance += money; // 可以加入校验逻辑  
        }  
    }  
}
```

典型真题演练

题 1 (单选)：封装的核心思想是将类的属性（），对外提供（）方法。

- A. public 修饰；getter/setter
- B. private 修饰；getter/setter
- C. protected 修饰；toString
- D. static 修饰；main

答案：B

题 2 (判断)：封装的主要目的是让外部可以直接访问类的内部属性。（）

答案：X。封装将属性私有化，对外提供公共方法访问，目的是隐藏内部实现、保护数据安全。

2.3 ★ 继承

✦ 应试速记

考点	速记
关键字	extends
单/多继承	单继承 （一个子类只有一个直接父类）
子类能继承什么	非私有 成员变量和方法
子类不能继承什么	私有成员、构造方法
子类创建时先调谁	先调父类构造 ，再调子类构造
super() 位置	子类构造方法 第一行

核心知识点

1. **继承使用 extends 关键字**
2. **Java 是单继承**：一个子类只能有一个直接父类
3. **子类可以继承**：父类的非私有成员变量和方法
4. **子类不能继承**：私有成员、构造方法
5. **多层继承**：A → B → C（C 可继承 A 的成员，但只能直接继承 B）
6. 子类创建时，**先调用父类构造方法**，再调用子类构造方法

super 关键字：

- super()：调用父类构造方法（必须在子类构造方法**第一行**）
- super.成员变量：访问父类成员变量
- super.方法名()：调用父类方法

```

class Animal {
    String name;
    public Animal(String name) {
        this.name = name;
    }
    public void eat() {
        System.out.println(name + "正在吃");
    }
}

class Dog extends Animal {
    public Dog(String name) {
        super(name); // 必须第一行调用父类构造
    }
    @Override
    public void eat() {
        super.eat(); // 调用父类方法
        System.out.println("狗在啃骨头");
    }
}

```

★ 重载（Overload） vs 重写（Override）

对比项	重载（Overload）	重写（Override）
发生位置	同一个类中	父子类之间
方法名	相同	相同
参数列表	必须不同 （个数/类型/顺序）	必须相同
返回值	无关（可相同可不同）	必须相同或兼容（子类）
权限	无关	不能比父类更严格
静态方法	可以重载	不能被重写
绑定时机	编译期（静态多态）	运行期（动态多态）

```
// 重载示例
class Calculator {
    public int add(int a, int b) { return a + b; }
    public double add(double a, double b) { return a + b; }    // 类型不同
    public int add(int a, int b, int c) { return a + b + c; }    // 个数不同
}

// 重写示例
class Parent {
    protected void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    @Override
    public void show() { System.out.println("Child"); }    // 权限 protected → public（正确）
}
```

典型真题演练

题 1（单选）：以下关于继承的描述，错误的是（ ）

- A. Java 允许一个子类继承多个父类
- B. 子类可作为另一个子类的父类
- C. 实例化子类时会调用父类构造方法
- D. 子类可以访问父类的公共属性

答案：A。Java 类是单继承，一个子类只能有一个直接父类。

题 2（单选）：下列选项中，关于方法重写说法正确的是（ ）

- A. 重写可以发生在同一个类中
- B. 重写方法的参数列表可以不同
- C. 重写方法的访问权限不能比父类更严格
- D. 静态方法可以被重写

答案：C

题 3（判断）：子类可以继承父类的构造方法。（ ）

答案：×。构造方法不能被继承，只能通过 `super()` 调用。

题 4（代码输出）：


```
class A {  
    public A() {  
        System.out.print("A");  
    }  
}  
  
class B extends A {  
    public B() {  
        System.out.print("B");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        new B();  
    }  
}
```

答案：AB

解析：创建子类对象时，先调用父类构造方法，再调用子类构造方法。

题 5（简答）：子类重写父类方法时需要注意哪些规则？

参考答案：

1. 方法名、参数列表必须与父类被重写的方法完全一致。
2. 返回值类型必须相同或是父类返回值的子类型。
3. 访问权限不能比父类方法更严格（如父类是 public，子类不能是 protected）。
4. 不能抛出比父类更宽泛的异常。
5. 静态方法、private 方法、final 方法不能被重写。

题 6（单选）：以下关于重载（Overload）的说法正确的是（ ）

- A. 重载必须发生在父子类之间
- B. 重载方法的参数列表必须不同
- C. 重载方法的返回值类型必须不同
- D. 重载方法的访问权限必须相同

答案：B

解析：重载发生在同一个类中；参数列表必须不同（个数/类型/顺序）；返回值与重载无关。

题 7（单选）：在 Java 中，子类可以继承父类的（ ）

- A. 所有成员变量和方法
- B. 非私有成员变量和方法

C. 私有成员变量和方法

D. 构造方法

答案：B

题 8（程序分析-输出结果）：

```
class Parent {  
    int x = 10;  
}  
class Child extends Parent {  
    int x = 20;  
    public void show() {  
        System.out.print(super.x + "," + this.x);  
    }  
}  
public class Test {  
    public static void main(String[] args) {  
        Child c = new Child();  
        c.show();  
    }  
}
```

答案：10,20

解析：super.x 访问父类的 x=10；this.x 访问子类的 x=20。

题 9（程序分析-输出结果）：

```

class A {
    public A() { System.out.print("A"); }
}
class B extends A {
    public B() {
        super();
        System.out.print("B");
    }
}
class C extends B {
    public C() {
        super();
        System.out.print("C");
    }
}
public class Test {
    public static void main(String[] args) {
        new C();
    }
}

```

答案：ABC

解析：多层继承时构造调用顺序：先父类再子类，逐层调用。

2.4 ★ 多态

✦ 应试速记

考点	速记
多态三条件	继承（实现接口）+ 重写 + 父类引用指向子类对象
向上转型	Parent p = new Child();
调用方法	执行 子类重写 的方法（动态绑定）
能否调用子类特有方法	✗ 不能
instanceof	判断对象是否属于某类型
向下转型	Child c = (Child) p; （需先 instanceof 判断）

核心知识点

1. 多态的定义：同一个方法在不同对象上表现出不同行为。

2. 实现多态的三个条件：

1. 继承（或实现接口）
2. 方法重写
3. 父类引用指向子类对象（向上转型）

3. 向上转型：

```
Animal a = new Dog(); // 父类引用指向子类对象
a.eat();               // 调用子类重写的方法（动态绑定）
// a.bark();           // 编译错误！不能调用子类特有方法
```

特点：

- 只能调用**父类中声明过**的方法
- 实际执行的是**子类重写**的逻辑（运行时多态）
- 不能调用子类特有的方法

4. 向下转型（需要强制转换）：

```
Animal a = new Dog();
if (a instanceof Dog) { // 先判断类型
    Dog d = (Dog) a;    // 向下转型
    d.bark();            // 可以调用子类特有方法
}
```

2.4.1 ★ 接口

接口定义：

```
interface Flyable {
    int MAX_SPEED = 100; // public static final 常量
    void fly(); // public abstract 方法（JDK8前）

    // JDK8 后可以有默认方法和静态方法
    default void show() { System.out.println("接口默认方法"); }
    static void info() { System.out.println("接口静态方法"); }
}
```

接口规则：

- 1. 关键字 interface ， 无构造方法
- 2. 成员变量默认 public static final
- 3. 方法默认 public abstract （JDK8前）
- 4. 类用 implements 实现接口， 可以实现多个接口
- 5. 接口可以继承接口 （ extends ），支持多继承

接口 vs 抽象类（简答题必考）：

对比项	抽象类	接口
关键字	abstract class	interface
构造方法	可以有	不能有
成员变量	任意权限	public static final
普通方法	可以有	JDK8前不能有，之后可有 default/static
抽象方法	可以有	可以有
单继承/多实现	单继承	多实现
设计思想	is-a 关系（共性抽取）	like-a 关系（功能扩展）

规定接口的好处：降低耦合、定义规范、便于扩展、实现多继承效果。

典型真题演练

题 1（单选）：多态性主要通过什么实现？（ ）
A. 方法重载 B. 类的继承 C. 抽象类 D. 方法重写和向上转型

答案：D

题 2（单选）：以下代码运行结果是（）

```
class Parent {  
    void show() { System.out.println("Parent"); }  
}  
class Child extends Parent {  
    void show() { System.out.println("Child"); }  
}  
public class Test {  
    public static void main(String[] args) {  
        Parent p = new Child();  
        p.show();  
    }  
}
```

A. Parent B. Child C. 编译错误 D. 运行时异常

答案：B

解析：向上转型后，调用的仍是子类重写的方法（动态绑定）。

题 3（简答）：什么是向上转型？向上转型在实现多态中起到了什么作用？

参考答案：

向上转型指**父类引用指向子类对象**，格式为 父类类型 变量名 = new 子类();。

作用：是多态实现的基础，让同一个父类引用可以指向不同的子类对象，调用同一个方法时表现出不同的行为，提高了代码的扩展性和灵活性。

题 4（简答）：请简述抽象类和接口的区别。（高频考点）

参考答案：（见上表，分点作答）

题 5（单选）：以下关于接口的说法错误的是（）

- A. 接口中所有方法都是抽象的
- B. 一个类可以实现多个接口
- C. 接口中变量默认是 public static final
- D. 接口不能实例化

答案：A

解析：JDK8 开始，接口可以有 default 方法和 static 方法（有方法体），不全是抽象的。

题 6 (判断): 抽象类可以没有抽象方法, 但包含抽象方法的类必须声明为抽象类。()

答案: ✓

题 7 (判断): 一个类实现了接口但没有实现所有抽象方法, 这个类必须声明为抽象类。()

答案: ✓

题 8 (填空): Java 中实现多态的机制是方法重写和_____。

答案: 向上转型 (或父类引用指向子类对象)

题 9 (单选): 以下关于向上转型的说法, 错误的是 ()

- A. 父类引用可以指向子类对象
- B. 通过父类引用可以调用子类特有的方法
- C. 通过父类引用调用重写方法时会执行子类的逻辑
- D. 向上转型是多态实现的基础

答案: B

解析: 向上转型后, 父类引用只能调用父类中声明过的方法, 不能调用子类特有方法。

题 10 (程序分析-输出结果):

```
interface A {  
    void show();  
}  
class B implements A {  
    public void show() { System.out.print("B"); }  
    public void test() { System.out.print("Test"); }  
}  
public class Test {  
    public static void main(String[] args) {  
        A a = new B();  
        a.show();  
        // a.test(); // 取消注释会怎样?  
    }  
}
```

答案: B。取消注释 a.test() 会编译错误, 因为接口 A 类型的引用只能调用接口中声明的方法, test() 是 B 的特有方法。

题 11 (程序分析-输出结果):

```

abstract class Animal {
    abstract void sound();
}
class Dog extends Animal {
    void sound() { System.out.println("汪汪"); }
}
class Cat extends Animal {
    void sound() { System.out.println("喵喵"); }
}
public class Test {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();
        a1.sound();
        a2.sound();
    }
}

```

答案：汪汪；喵喵

解析：多态的体现——同一父类引用指向不同子类对象，调用同一方法表现不同行为。

2.5 抽象类

核心知识点

```

public abstract class Animal {
    private String name;

    public Animal(String name) { this.name = name; } // 构造方法

    public abstract void eat(); // 抽象方法，无方法体

    public void sleep() { // 普通方法
        System.out.println(name + "在睡觉");
    }
}

```

规则总结：

- 1. abstract 声明，不能 new 创建对象
- 2. 可以包含：成员变量、构造方法、普通方法、抽象方法、静态方法
- 3. 抽象方法：只有声明没有方法体， public abstract void eat();
- 4. 普通子类继承抽象类，必须重写所有抽象方法
- 5. 抽象子类继承抽象父类，可以不重写抽象方法

典型真题演练

题 1（判断）：抽象类中可以包含构造方法。（）

答案：√。抽象类可以有构造方法，供子类通过 super() 调用。

题 2（单选）：以下关于抽象类的描述，正确的是（）

- A. 抽象类不能有成员变量
- B. 抽象类中必须包含抽象方法
- C. 抽象类不能实例化
- D. 抽象类中的方法都是抽象方法

答案：C
解析：A 抽象类可以有成员变量；B 抽象类可以不包含抽象方法；D 抽象类可以有普通方法。

2.6 内部类（了解，选择题）

类型	说明
成员内部类	定义在类中的普通类，可以访问外部类所有成员
静态内部类	static 修饰的内部类，只可访问外部类静态成员
局部内部类	定义在方法中的类，作用域仅限于方法内
匿名内部类	没有名字的内部类，常用于简化事件监听等（Lambda 底层原理）

```
// 匿名内部类示例
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("运行中");
    }
};

// Lambda 简化（JDK8+）
Runnable r = () -> System.out.println("运行中");
```

典型真题演练

题 1（单选）：以下哪个是 Lambda 表达式的底层实现原理？（）
A. 成员内部类 B. 静态内部类 C. 局部内部类 D. 匿名内部类

答案：D

✂ 模块三 常用 API

3.1 Math 类

特点：构造方法私有（不能 new），所有方法都是静态的，通过 Math.方法名() 调用

方法	说明	示例
abs(int a)	绝对值	Math.abs(-5) → 5
ceil(double a)	向上取整	Math.ceil(3.1) → 4.0
floor(double a)	向下取整	Math.floor(3.9) → 3.0
round(double a)	四舍五入	Math.round(3.5) → 4
random()	返回 [0.0, 1.0) 随机数	Math.random()
max(a, b)	最大值	Math.max(3, 5) → 5
min(a, b)	最小值	Math.min(3, 5) → 3
pow(a, b)	a 的 b 次幂	Math.pow(2, 3) → 8.0

生成 [min, max] 范围内的随机整数：

```
int randomNum = (int)(Math.random() * (max - min + 1)) + min;
// 例如 [1, 100]: (int)(Math.random() * 100) + 1
```

3.2 System 类

方法	说明
currentTimeMillis()	当前时间戳（毫秒）
arraycopy(源, 源起点, 目标, 目标起点, 长度)	数组复制
gc()	通知垃圾回收器
exit(int status)	终止 JVM（finally 中不会执行）

3.3 ★ 日期时间类（LocalDate 等）

Java 8 新的日期时间 API（java.time 包）：

类	说明
LocalDate	日期（年月日）
LocalTime	时间（时分秒纳秒）
LocalDateTime	日期 + 时间
DateTimeFormatter	格式化/解析
Duration	时间间隔（时分秒）
Period	日期间隔（年月日）

```
// 获取当前日期时间
LocalDate today = LocalDate.now();           // 2026-07-03
LocalTime now = LocalTime.now();             // 14:30:00
LocalDateTime dt = LocalDateTime.now();      // 2026-07-03T14:30:00

// 指定日期
LocalDate date = LocalDate.of(2024, 12, 25); // 2024-12-25

// 常用操作
LocalDate tomorrow = today.plusDays(1);      // 加一天
LocalDate lastMonth = today.minusMonths(1);   // 减一个月
int year = today.getYear();                  // 获取年份
int month = today.getMonthValue();           // 获取月份（1-12）
int day = today.getDayOfMonth();             // 获取日

// 格式化
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");
String formatted = today.format(formatter);   // "2026-07-03"

// 解析
LocalDate parsed = LocalDate.parse("2024-06-15"); // 默认格式 yyyy-MM-dd

// 日期间隔
Period period = Period.between(LocalDate.of(2020, 1, 1), today);
// → P6Y6M2D（6年6个月2天）
```

3.4 ★ 包装类

基本类型 vs 包装类：

基本类型	包装类
int	Integer
char	Character
double	Double
boolean	Boolean
...	...

自动装箱与拆箱：

```
Integer i = 100;           // 自动装箱：int → Integer（实际调用 Integer.valueOf(100)）
int n = i;                 // 自动拆箱：Integer → int（实际调用 i.intValue()）
```

⚠ Integer 缓存池：

```
Integer a = 127;
Integer b = 127;
System.out.println(a == b);    // true（缓存池范围内，复用对象）

Integer c = 128;
Integer d = 128;
System.out.println(c == d);    // false（超出 -128~127 范围，新建对象）
```

常用方法：

方法	说明
Integer.parseInt("123")	String → int
Integer.valueOf(123)	int → Integer
String.valueOf(123)	int → String
"123".toString()	int → String

3.5 Random 类

```
Random r = new Random();
int num = r.nextInt(100);    // 生成 [0, 100) 的随机整数
int num2 = r.nextInt(10) + 1; // 生成 [1, 10]
```

典型真题演练

题 1（单选）：Math.round(11.5) 的结果是（ ）

- A. 11 B. 11.5 C. 12 D. 12.0

答案：C

解析：四舍五入， $11.5 \rightarrow 12$ 。

题 2（单选）：以下代码输出结果是（）

```
Integer a = 100;
Integer b = 100;
Integer c = 200;
Integer d = 200;
System.out.println((a == b) + "," + (c == d));
```

A. true,true B. true,false C. false,true D. false,false

答案：B

解析：Integer 缓存池范围 -128~127，100 在缓存池内故 ab 为 true；200 超出范围创建新对象故 cd 为 false。

题 3（填空）：Math 类中用于生成 [0.0, 1.0) 范围内随机数的方法是_____。

答案： random()

题 4（代码输出）：

```
LocalDate date1 = LocalDate.of(2024, 1, 1);
LocalDate date2 = date1.plusDays(10);
System.out.println(date2);
```

答案： 2024-01-11

题 5（填空）：将字符串 "123" 转换为 int 类型的方法是_____。

答案： Integer.parseInt("123")

题 6（编程）：使用 Math.random() 生成一个 1~100（含）之间的随机整数。

参考代码： int num = (int)(Math.random() * 100) + 1;

题 7（单选）：下列哪个日期时间类可以表示"2024-06-15"这种格式？（）

A. LocalTime B. LocalDateTime C. LocalDate D. Date

答案：C

题 8（判断）： Math.floor(-3.5) 的结果是 -3。（）

答案：X。Math.floor(-3.5) = -4.0 （向下取整，往小取）

题 9（代码输出）：

```
int x = Integer.parseInt("10");
double y = Double.parseDouble("3.14");
System.out.println(x + y);
```

答案：13.14

题 10（单选）：以下关于包装类的说法，正确的是（）

- A. Integer a = 128; Integer b = 128; a == b 结果为 true
- B. 包装类可以直接进行算术运算
- C. 包装类不能与基本类型相互转换
- D. 自动拆箱是指将包装类对象自动转换为基本类型

答案：D

解析：A 128 超出缓存池范围，== 为 false；B 包装类参与运算时会自动拆箱；C 可以自动装箱拆箱。

题 11（代码输出）：

```
System.out.println(Math.abs(-5));
System.out.println(Math.ceil(3.1));
System.out.println(Math.floor(3.9));
```

答案：5; 4.0; 3.0

题 12（程序分析-输出结果）：

```
public class Test {
    public static void main(String[] args) {
        Integer i1 = 127;
        Integer i2 = 127;
        Integer i3 = new Integer(127);
        System.out.println(i1 == i2);    // ?
        System.out.println(i1 == i3);    // ?
        System.out.println(i1.equals(i3)); // ?
    }
}
```

答案：true；false；true

解析：i1和i2在缓存池范围内复用对象；i3是new出来的新对象，地址不同；equals比较内容相同。

模块四 集合框架

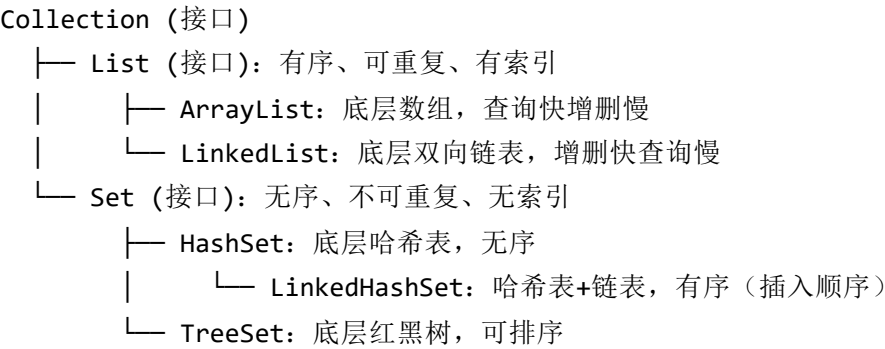
4.1 泛型

作用：编译期类型校验，避免强制类型转换异常

```
List<String> list = new ArrayList<>(); // 只能存储 String
// list.add(123);                      // 编译错误!
String s = list.get(0);                // 不需要强制类型转换
```

4.2 ★ 单列集合（Collection）

继承体系



ArrayList vs LinkedList

对比项	ArrayList	LinkedList
底层结构	动态数组	双向链表
查询速度	快（随机访问 O(1)）	慢（需遍历 O(n)）
增删速度	慢（需移动元素）	快（修改指针）
内存占用	连续空间，预分配容量	每个节点存储前后指针

对比项	ArrayList	LinkedList
适用场景	频繁查询、很少增删	频繁增删、很少查询

Set 集合特点

集合	底层	特点	顺序
HashSet	哈希表 (HashMap)	去重依赖 hashCode()+equals()	无序
LinkedHashSet	哈希表+双向链表	去重+保序	插入顺序
TreeSet	红黑树	自动排序	排序后顺序

★ TreeSet 自定义排序

方式一：元素类实现 Comparable 接口，重写 compareTo()

```
class Student implements Comparable<Student> {
    String name;
    int score;

    @Override
    public int compareTo(Student o) {
        return this.score - o.score; // 按成绩升序
    }
}

TreeSet<Student> ts = new TreeSet<>();
```

方式二：创建 TreeSet 时传入 Comparator 比较器

```
TreeSet<Student> ts = new TreeSet<>(new Comparator<Student>() {
    @Override
    public int compare(Student o1, Student o2) {
        return o1.score - o2.score;
    }
});
// Lambda 简化：
TreeSet<Student> ts = new TreeSet<>((o1, o2) -> o1.score - o2.score);
```

4.3 ★ 双列集合 Map

继承体系

Map (接口)

- └─ HashMap: 底层哈希表, 允许 null 键/值, 无序
- └─ LinkedHashMap: 哈希表+链表, 有序 (插入顺序)
- └─ TreeMap: 红黑树, 键可排序

特点:

- 键值对 (K-V) 形式存储
- 键唯一, 值可重复
- 键和值都可以是任意引用类型

4.4 集合遍历

List 遍历方式:

```
List<String> list = new ArrayList<>();  
// 1. 普通 for  
for (int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i));  
}  
// 2. 增强 for  
for (String s : list) {  
    System.out.println(s);  
}  
// 3. 迭代器  
Iterator<String> it = list.iterator();  
while (it.hasNext()) {  
    System.out.println(it.next());  
}
```

Map 遍历方式:

```
Map<String, Integer> map = new HashMap<>();
// 1. keySet() 遍历键
for (String key : map.keySet()) {
    System.out.println(key + "=" + map.get(key));
}
// 2. entrySet() 遍历键值对
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + "=" + entry.getValue());
}
```

⚠️ **集合遍历时删除元素**：必须使用迭代器的 `remove()` 方法，否则会报 `ConcurrentModificationException`。

典型真题演练

题 1（单选）：以下哪个集合类不允许有重复元素，并且元素顺序是插入顺序？（）

A. HashSet B. TreeSet C. LinkedHashSet D. LinkedList

答案：C

解析：LinkedHashSet 基于哈希表+链表，去重且保持插入顺序。HashSet 无序，TreeSet 排序，LinkedList 允许重复。

题 2（多选）：下列关于集合类描述正确的是（）

A. ArrayList 和 LinkedList 均实现了 List 接口
B. ArrayList 的查询速度比 LinkedList 快
C. 添加和删除元素时，ArrayList 的表现更佳
D. HashMap 实现 Map 接口，允许 null 用作键或值

答案：ABD

解析：ArrayList 增删慢（C 错误），因为需要移动元素。

题 3（单选）：HashSet 判断元素重复的依据是（）

A. 只比较 equals()
B. 只比较 hashCode()
C. 先比较 hashCode()，再比较 equals()
D. 使用 == 比较

答案：C

题 4 (填空): TreeSet 中实现自定义排序的两种方法是：元素类实现_____接口，或在创建 TreeSet 时传入_____。

答案：Comparable；Comparator

题 5 (代码输出):

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
for (String s : list) {
    list.remove(s); // 会怎样?
}
```

答案：抛出 ConcurrentModificationException （并发修改异常）

解析：增强 for 遍历时不能直接使用集合的 remove 方法，应使用迭代器的 remove。

题 6 (填空): Map 集合遍历时，使用_____方法获取所有键的集合，使用_____方法获取所有键值对的集合。

答案：keySet()；entrySet()

题 7 (简答): 请简述 ArrayList 和 LinkedList 的区别。

参考答案：

1. 底层结构不同：ArrayList 基于动态数组，LinkedList 基于双向链表。
2. 查询性能不同：ArrayList 随机访问快 ($O(1)$)，LinkedList 慢 ($O(n)$)。
3. 增删性能不同：ArrayList 增删慢（需移动元素），LinkedList 快（修改指针）。
4. 内存占用不同：ArrayList 连续空间，LinkedList 需额外存储前后指针。
5. 适用场景不同：频繁查询用 ArrayList，频繁增删用 LinkedList。

题 8 (单选): 以下哪个集合类底层使用红黑树实现？（）

A. ArrayList B. HashSet C. TreeSet D. LinkedList

答案：C

题 9 (单选): 关于 HashMap 的描述，错误的是（）

- A. 键值对存储
- B. 键不能重复
- C. 线程安全
- D. 允许 null 键和 null 值

答案：C

解析：HashMap 是非线程安全的。线程安全的 Map 是 Hashtable 或 ConcurrentHashMap。

题 10（程序分析-输出结果）：

```
import java.util.*;
public class Test {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
        set.add("B");
        set.add("A");
        set.add("C");
        set.add("A"); // 重复
        System.out.println(set.size());
    }
}
```

答案：3

解析：HashSet 去重，重复的 "A" 不会添加，最终只有 "B"、"A"、"C" 三个元素。

题 11（程序分析-输出结果）：

```
import java.util.*;
public class Test {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("张三", 85);
        map.put("李四", 92);
        map.put("张三", 95); // 覆盖
        System.out.println(map.get("张三"));
        System.out.println(map.size());
    }
}
```

答案：95；2

解析：Map 键唯一，第二次 put("张三",95) 会覆盖第一次的值。size=2。

⚠ 模块五 异常处理

5.1 异常体系

Throwable



5.2 异常处理机制

try-catch-finally

```
try {  
    // 可能出异常的代码  
    int result = 10 / 0;  
} catch (ArithmeticException e) {           // 子类异常在前  
    System.out.println("除数不能为0");  
} catch (Exception e) {                     // 父类异常在后  
    e.printStackTrace();  
} finally {  
    System.out.println("无论是否异常都执行"); // 释放资源  
}
```

重要规则:

1. 多个 catch, 子类在前, 父类在后
2. finally 总执行, 除非 JVM 退出 (`System.exit(0)`)
3. finally 中如果有 return, 会覆盖 try/catch 中的 return

throws 声明异常

```
public void readFile(String path) throws IOException, ParseException {  
    // 方法内不处理，交给调用者处理  
}
```

throw 抛出异常

```
public void setAge(int age) {  
    if (age < 0 || age > 150) {  
        throw new IllegalArgumentException("年龄不合法");  
    }  
}
```

★ throw vs throws（简答题高频）

对比项	throw	throws
位置	方法内部	方法声明处
内容	抛出一个具体的异常对象	声明可能抛出的异常类型
数量	一次抛一个	可声明多个（逗号分隔）
谁处理	由方法内代码处理或继续抛出	交给调用者处理

典型真题演练

题 1（单选）：下列关于 Java 异常处理的说法，不正确的是（ ）

- A. try 块中可以匹配多个 catch 块
- B. finally 块中的代码总是会执行，除非 JVM 退出
- C. throw 语句用于抛出异常对象
- D. throws 关键字声明的异常必须在方法内部被捕获

答案：D

解析：throws 声明异常是交给调用者处理，不是必须在方法内捕获。

题 2（单选）：以下哪个异常属于运行时异常（RuntimeException）？（ ）

- A. IOException B. SQLException C. NullPointerException D. ParseException

答案：C

题 3（代码输出）：

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            System.out.print("A");  
            int a = 10 / 0;  
            System.out.print("B");  
        } catch (Exception e) {  
            System.out.print("C");  
        } finally {  
            System.out.print("D");  
        }  
    }  
}
```

答案：ACD

解析：A 输出，然后除 0 异常跳转 catch 输出 C，finally 总执行输出 D。B 不会执行。

题 4（简答）：请简述 throw 和 throws 的区别。

参考答案：（见上表，分点作答）

题 5（代码输出）：

```
public class Test {  
    public static int test() {  
        try {  
            return 1;  
        } finally {  
            return 2;  
        }  
    }  
    public static void main(String[] args) {  
        System.out.println(test());  
    }  
}
```

答案：2

解析：finally 中的 return 会覆盖 try 中的 return。（⚠️ 容易出错，实际开发应避免）

题 6 (单选)：以下哪个关键字用于手动抛出异常？（）

A. throws B. throw C. try D. catch

答案：B

题 7 (判断)：NullPointerException 是受检异常（编译时异常）。（）

答案：×。NullPointerException 是运行时异常（非受检异常）。

题 8 (单选)：以下关于 finally 块的说法，正确的是（）

- A. 异常发生时才执行
- B. 异常未发生时就不执行
- C. 无论是否发生异常都执行
- D. 只有 catch 块中有 return 时才执行

答案：C

解析：finally 块无论是否发生异常都会执行，常用于释放资源。

题 9 (程序分析-输出结果)：

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int[] arr = {1, 2, 3};  
            System.out.print(arr[5]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.print("越界");  
        } catch (Exception e) {  
            System.out.print("异常");  
        } finally {  
            System.out.print("结束");  
        }  
    }  
}
```

答案：越界结束

解析：访问数组越界抛出 ArrayIndexOutOfBoundsException，被第一个 catch 捕获，最后执行 finally。

模块六 IO 流

6.1 File 类

作用：文件和目录路径的抽象表示（**不负责读写文件内容**）

常用方法：

方法	说明
exists()	文件或目录是否存在
createNewFile()	创建新文件
mkdir()	创建目录
delete()	删除文件或空目录
list()	返回当前目录下所有文件名（String[]）
listFiles()	返回当前目录下所有文件对象（File[]）
getAbsolutePath()	获取绝对路径
isFile()	判断是否是文件
isDirectory()	判断是否是目录
length()	文件大小（字节）

 **删除目录：**只能删除空目录，非空目录需要**递归删除**。

6.2 流的分类

分类标准	类别
数据流向	输入流（读）、输出流（写）
数据单位	字节流（8位）、字符流（16位）
功能层次	节点流（直接操作数据源）、处理流（装饰节点流）

6.3 ★ 字节流

InputStream (抽象类)

- └─ FileInputStream (文件字节输入流)
- └─ BufferedInputStream (缓冲字节输入流)

OutputStream (抽象类)

- └─ FileOutputStream (文件字节输出流)
- └─ BufferedOutputStream (缓冲字节输出流)

适用场景：处理**二进制文件**（图片、视频、音频等）

文件拷贝模板（编程题高频）：

```
try {
    BufferedInputStream bis = new BufferedInputStream(new FileInputStream("源路径"));
    BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream("目标路径"));
    byte[] buf = new byte[1024];
    int len;
    while ((len = bis.read(buf)) != -1) {
        bos.write(buf, 0, len);
    }
    bis.close();
    bos.close();
} catch (IOException e) {
    e.printStackTrace();
}
```

6.4 ★ 字符流

Reader (抽象类)

- └─ FileReader (文件字符输入流)
- └─ BufferedReader (缓冲字符输入流，提供 readLine())

Writer (抽象类)

- └─ FileWriter (文件字符输出流)
- └─ BufferedWriter (缓冲字符输出流，提供 newLine())

适用场景：处理**纯文本文件**

```
// 按行读取文本文件
BufferedReader br = new BufferedReader(new FileReader("input.txt"));
String line;
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
br.close();

// 写入文本文件
BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));
bw.write("Hello World");
bw.newLine(); // 换行
bw.write("第二行");
bw.close();
```

6.5 转换流

类	说明
InputStreamReader	字节输入流 → 字符输入流（可指定编码）
OutputStreamWriter	字节输出流 → 字符输出流（可指定编码）

```
// 指定编码读取
BufferedReader br = new BufferedReader(
    new InputStreamReader(new FileInputStream("file.txt"), "UTF-8"));
```

典型真题演练

题 1（单选）：以下哪个方法用于判断 File 对象对应的文件是否存在？（ ）

A. isFile() B. exists() C. createNewFile() D. isDirectory()

答案：B

题 2（单选）：以下哪个流类适用于读取文本文件并按行处理？（ ）

A. FileInputStream B. FileOutputStream C. BufferedReader D. BufferedOutputStream

答案：C

解析：BufferedReader 的 readLine() 方法可以一次读一行。

题 3 (单选)：字节流和字符流的根本区别是 ()

- A. 字节流处理二进制，字符流处理文本
- B. 字节流以字节为单位，字符流以字符为单位
- C. 字节流效率高，字符流效率低
- D. 字节流有缓冲，字符流没有

答案：B

题 4 (填空)：File 类的_____方法用于判断路径是否为目录。

答案：isDirectory()

题 5 (编程)：使用字节缓冲流实现文件拷贝的核心代码。(高频考题)

参考代码：(见上文文件拷贝模板)

题 6 (判断)：File 类既可以操作文件路径，也可以读写文件内容。()

答案：X。File 类只负责文件/目录路径的抽象表示，不负责读写内容。读写内容使用 IO 流。

题 7 (单选)：以下哪个是字节输入流的顶层抽象类？ ()

- A. InputStream
- B. OutputStream
- C. Reader
- D. Writer

答案：A

题 8 (程序分析-输出结果)：

```
import java.io.*;
public class Test {
    public static void main(String[] args) throws IOException {
        File file = new File("test.txt");
        System.out.print(file.exists());           // 假设文件不存在
        file.createNewFile();
        System.out.print(file.exists());
        file.delete();
        System.out.print(file.exists());
    }
}
```

答案：false>true/false

解析：初始不存在→创建后存在→删除后又不存在。

题 9（填空）：字节缓冲流中的 `BufferedInputStream` 的作用是_____。

答案：提高字节输入流的读写效率（提供缓冲区）

模块七 多线程

✦ 本节仅考选择题、填空题，了解核心概念即可，不需掌握编程细节。

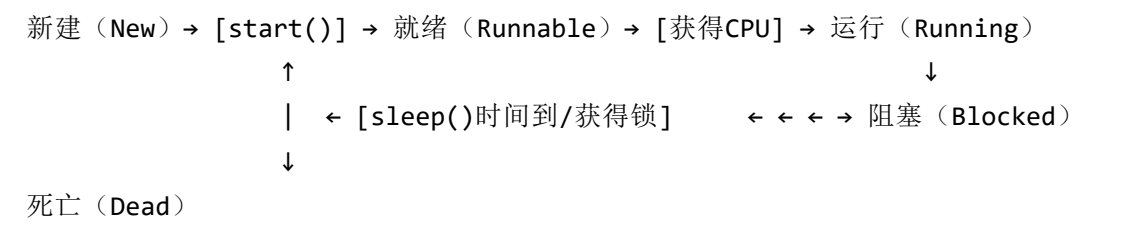
✦ 应试速记

考点	速记
进程 vs 线程	进程=资源分配单位，线程=CPU调度单位
创建方式1	继承 <code>Thread</code> ，重写 <code>run()</code> ，调用 <code>start()</code>
创建方式2	实现 <code>Runnable</code> ，重写 <code>run()</code> ，传给 <code>Thread</code> 再 <code>start()</code>
推荐方式	<code>Runnable</code> （避免单继承局限，适合共享资源）
启动线程	必须调用 <code>start()</code> ，不是 <code>run()</code>
线程状态	新建→就绪→运行→阻塞→死亡
同步关键字	<code>synchronized</code> （同步代码块/同步方法）
<code>sleep</code> vs <code>wait</code>	<code>sleep</code> 不释放锁 ， <code>wait</code> 释放锁

7.1 线程的两种创建方式

方式	优点	缺点
继承 <code>Thread</code>	代码简洁，可直接使用 <code>Thread</code> 方法	Java 单继承局限
实现 <code>Runnable</code>	避免单继承，适合多线程共享资源，耦合度低	代码略复杂

7.2 线程生命周期



7.3 线程同步

- 问题：多个线程同时操作共享资源 → 数据不一致
- 解决：synchronized 关键字
 - 同步代码块：synchronized(锁对象) { }
 - 同步方法：public synchronized void method() { }

7.4 sleep() vs wait()

对比项	sleep()	wait()
所属类	Thread 的静态方法	Object 的实例方法
是否释放锁	不释放	释放
调用条件	任何地方	必须在同步代码块/方法中
唤醒方式	时间到自动唤醒	notify() / notifyAll()

典型真题演练

题 1（单选）：以下哪个关键字用于线程同步？（ ）

A. static B. synchronized C. final D. volatile

答案：B

题 2（单选）：启动一个新线程必须调用（ ）

A. run() B. start() C. stop() D. exit()

答案：B

题 3（单选）：以下哪个方法让线程进入休眠状态？（）

- A. yield() B. stop() C. sleep() D. wait()

答案：C

题 4（判断）：直接调用线程对象的 run() 方法会启动一个新线程。（）

答案：X。直接调用 run() 只是普通方法调用，必须调用 start() 才启动新线程。

题 5（单选）：调用 start() 后线程进入的状态是（）

- A. 新建 B. 就绪 C. 运行 D. 阻塞

答案：B。start() 使线程进入就绪状态，等待 CPU 调度。

题 6（单选）：sleep() 和 wait() 的区别中，正确的是（）

- A. sleep() 释放锁，wait() 不释放
B. sleep() 不释放锁，wait() 释放锁
C. 两者都不释放锁
D. 两者都释放锁

答案：B

题 7（填空）：解决线程安全问题可以使用_____关键字。

答案：synchronized

📖 模块八 GUI

📌 本节仅考选择题、填空题，了解核心组件、布局和事件即可，不需掌握编程细节。

📌 应试速记

考点	速记
顶层窗口	JFrame
JFrame 默认布局	BorderLayout
JFrame 默认可见性	不可见 ，需 setVisible(true)

考点	速记
流式布局	FlowLayout ， 从左到右排列
边界布局	BorderLayout ， 东南西北中五区域
网格布局	GridLayout ， 行列均分
按钮点击监听	ActionListener
窗口关闭监听	WindowListener

8.1 常用组件

组件	类名	说明
窗口	JFrame	顶级容器
对话框	JDialog	弹出对话框
标签	JLabel	显示文本/图片
按钮	JButton	点击触发事件
文本框	JTextField	单行输入
面板	JPanel	中间容器

8.2 布局管理器

布局	类名	特点
流式布局	FlowLayout	从左到右排列，放不下自动换行
边界布局	BorderLayout	分 EAST/SOUTH/WEST/NORTH/CENTER（JFrame 默认）
网格布局	GridLayout	按行列排列，每个组件大小相同

8.3 事件监听

监听器	事件	方法
ActionListener	按钮点击	actionPerformed(ActionEvent e)
MouseListener	鼠标操作	click/press/release/enter/exit

监听器	事件	方法
WindowListener	窗口操作	open/close/activate
KeyListener	键盘操作	press/release/type

典型真题演练

题 1（单选）：处理窗口关闭事件使用（ ）
A. WindowListener B. ActionListener C. MouseListener D. KeyAdapter

答案：A

题 2（单选）：JFrame 的默认布局管理器是（ ）
A. FlowLayout B. BorderLayout C. GridLayout D. CardLayout

答案：B

题 3（单选）：设置 JFrame 窗口大小的方法是（ ）
A. setSize() B. setBounds() C. setVisible() D. setDefaultCloseOperation()

答案：A

题 4（填空）：要让 JFrame 显示，必须调用_____方法。

答案：setVisible(true)

题 5（判断）：JFrame 默认不可见，需调 setVisible(true) 才显示。（ ）

答案：✓



第二部分 期末模拟试卷（满分100分）

🕒 建议用时：120分钟
📋 题型分布：一、单选 15题×2分=30分 | 二、填空 10题×2分=20分 | 三、程序分析 3题×5分=15

一、单项选择题（共15题，每题2分，共30分）

1. Java 中，负责将 .java 源文件编译为 .class 字节码的是（ ）

- A. JVM B. JRE C. javac D. java

答案：C

解析：javac 是编译器，将 .java 编译为 .class；java 是运行工具；JVM 是运行时环境。

2. 以下哪个是 Java 的基本数据类型？（ ）

- A. String B. int C. Integer D. ArrayList

答案：B

解析：A 是引用类型，C 是包装类（引用类型），D 是集合类（引用类型）。

3. 执行以下代码后，输出结果是（ ）

```
String s1 = "Hello";  
String s2 = "Hello";  
System.out.println(s1 == s2);
```

- A. true B. false C. 编译错误 D. 运行时异常

答案：A

解析：两个字面量都指向常量池中的同一对象，故 == 比较地址为 true。

4. 以下关于方法重载的说法，正确的是（ ）

- A. 重载必须发生在父子类之间
B. 重载方法的返回值类型必须不同
C. 重载方法的参数列表必须不同
D. 重载方法的名称可以不同

答案：C

解析：重载发生在同一类中，方法名相同，参数列表不同，与返回值无关。

5. 以下关于 final 关键字的描述，错误的是（ ）

- A. final 修饰的变量不能重新赋值
B. final 修饰的方法不能被重写

- C. final 修饰的类不能被继承
- D. final 修饰的变量可以不初始化

答案：D

解析：final 修饰的变量必须在声明时或在构造方法中初始化，不能留空。

6. 以下哪个集合类元素有序、可重复、有索引？（）

- A. HashSet
- B. TreeSet
- C. ArrayList
- D. HashMap

答案：C

解析：List 接口的特点是 有序+可重复+有索引，ArrayList 是 List 的实现类。

7. Math.floor(-2.5) 的结果是（）

- A. -2.0
- B. -3.0
- C. -2
- D. -3

答案：B

解析：floor 向下取整，-2.5 向下（往更小方向）取为 -3.0。

8. 下列哪个异常是受检异常（Checked Exception）？（）

- A. NullPointerException
- B. ArrayIndexOutOfBoundsException
- C. ArithmeticException
- D. IOException

答案：D

解析：IOException 是受检异常，编译时必须处理。A、B、C 均为运行时异常。

9. 以下哪个关键字用于实现线程同步？（）

- A. volatile
- B. synchronized
- C. transient
- D. native

答案：B

10. 以下代码运行结果是（）

```

class A {
    void show() { System.out.print("A"); }
}
class B extends A {
    void show() { System.out.print("B"); }
}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        obj.show();
    }
}

```

A. A B. B C. 编译错误 D. 运行时异常

答案：B

解析：多态——父类引用指向子类对象，调用重写方法时执行子类逻辑。

11. 以下哪个是线程安全的 StringBuilder 替代方案？（）

A. String B. StringBuffer C. StringWriter D. StringReader

答案：B

解析：StringBuffer 方法是同步的（synchronized），线程安全。

12. 关于 HashMap 的描述，正确的是（）

A. 键和值都不能为 null
 B. 键唯一，值可重复
 C. 线程安全
 D. 有序存储

答案：B

解析：HashMap 键唯一（值可重复），允许 null 键和 null 值，非线程安全，无序。

13. 以下哪个是字符输出流的顶层抽象类？（）

A. InputStream B. OutputStream C. Reader D. Writer

答案：D

14. 下列代码的输出结果是（）

```
int i = 0;
if (i++ == 0 && i++ == 1) {}
System.out.println(i);
```

A. 1 B. 2 C. 0 D. 编译错误

答案：B

解析： $i++==0$ 先判断 ($i=0 \rightarrow \text{true}$)， i 自增为1；再判断 $i++==1$ ($i=1 \rightarrow \text{true}$)， i 自增为2。 $\&\&$ 两侧都执行。

15. 关于抽象类和接口，以下说法错误的是（ ）

- A. 抽象类可以有构造方法
- B. 接口中所有方法都是抽象的
- C. 一个类可以实现多个接口
- D. 抽象类用 `abstract` 关键字声明

答案：B

解析：JDK8 开始接口可以包含 `default` 和 `static` 方法（有方法体），不全是抽象的。

二、填空题（共10题，每题2分，共20分）

1. 一个 Java 源文件中最多可以定义_____个 `public` 类。

答案：1

2. Java 中所有类的根类是_____。

答案：`Object`

3. `String` 类中，用于比较两个字符串内容是否相等的方法是_____。

答案：`equals()`

4. 使用关键字_____可以声明常量，确保变量只能被赋值一次。

答案：`final`

5. 多态实现的基础是继承/接口实现、方法重写和_____。

答案：向上转型（父类引用指向子类对象）

6. 集合中 Collection 接口下，List 接口的特点是_____、可重复、有索引。

答案：有序

7. 在 Java 异常处理中，throws 关键字用在_____处声明可能抛出的异常。

答案：方法声明

8. 启动线程必须调用_____方法，而不是 run() 方法。

答案：start()

9. Math 类中用于生成一个 [0.0, 1.0) 范围内的随机数的方法是_____。

答案：random()

10. File 类中，判断文件或目录是否存在的方法是_____。

答案：exists()

三、程序分析题（共3题，每题5分，共15分）

1. 阅读以下程序，写出运行结果。

```
public class Test {  
    public static void change(int x) {  
        x = 100;  
    }  
    public static void main(String[] args) {  
        int a = 10;  
        change(a);  
        System.out.println(a);  
  
        StringBuilder sb = new StringBuilder("Hello");  
        sb.append(" World");  
        System.out.println(sb.toString());  
    }  
}
```

答案：

10

Hello World

解析:

(1) Java 是值传递, change(a) 将 a 的值 10 复制给形参 x, x=100 不影响原 a, a 仍为 10。

(2) StringBuilder 可变, append 直接修改原对象内容, 得到 "Hello World"。

2. 阅读以下程序, 写出运行结果。

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int[] arr = new int[5];  
            arr[10] = 100;  
            System.out.println("正常");  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("数组越界");  
        } catch (Exception e) {  
            System.out.println("其他异常");  
        } finally {  
            System.out.println("结束");  
        }  
        System.out.println("退出");  
    }  
}
```

答案:

数组越界

结束

退出

解析: arr[10] 超出数组范围[0~4], 抛出 ArrayIndexOutOfBoundsException, 被第一个 catch 捕获, 打印"数组越界"; finally 总执行; 程序继续执行后续代码。

3. 阅读以下程序, 写出运行结果。


```

class Animal {
    public void sound() {
        System.out.print("动物叫声 ");
    }
}

class Dog extends Animal {
    public void sound() {
        System.out.print("汪汪 ");
    }
    public void guard() {
        System.out.print("看门 ");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
        // a.guard(); // 该行若取消注释会怎样?

        if (a instanceof Dog) {
            Dog d = (Dog) a;
            d.guard();
        }
    }
}

```

答案：汪汪 看门

解析：

- (1) Animal a = new Dog() 向上转型，a.sound() 执行子类重写方法输出"汪汪"。
- (2) 若 a.guard() 取消注释 → 编译错误，父类引用不能调用子类特有方法。
- (3) instanceof 判断类型安全后向下转型，d.guard() 输出"看门"。

四、简答题（共3题，每题5分，共15分）

1. 请简述抽象类和接口的区别。

参考答案：

- ① 声明方式：抽象类用 `abstract class`，接口用 `interface`。
- ② 构造方法：抽象类可以有，接口不能有。

- ③ 成员变量：抽象类任意权限，接口默认 `public static final`。
- ④ 方法：抽象类可有普通方法+抽象方法；接口 JDK8 前只有抽象方法，JDK8 后可含 `default/static` 方法。
- ⑤ 继承/实现：类只能单继承抽象类，但可多实现接口。
- ⑥ 设计思想：抽象类 is-a 关系（共性抽取），接口 like-a 关系（功能扩展）。

2. 请简述 `throw` 和 `throws` 的区别。

参考答案：

- ① 位置不同：`throw` 在**方法内部**，`throws` 在**方法声明处**。
- ② 内容不同：`throw` 抛出**具体的异常对象**，`throws` 声明**异常类型**。
- ③ 数量不同：`throw` 一次抛一个，`throws` 可声明多个（逗号分隔）。
- ④ 处理方式：`throw` 后的语句不再执行；`throws` 将异常交给调用者处理。

3. 请简述 `String`、`StringBuilder`、`StringBuffer` 三者的区别。

参考答案：

- ① **可变性**：`String` 不可变，每次修改创建新对象；`StringBuilder` 和 `StringBuffer` 可变。
- ② **线程安全**：`String` 安全（不可变性）；`StringBuffer` 安全（方法同步）；`StringBuilder` 不安全。
- ③ **效率**：`String` 最低（频繁修改时），`StringBuilder` 最高，`StringBuffer` 居中。
- ④ **适用场景**：少量操作用 `String`；单线程大量修改用 `StringBuilder`；多线程用 `StringBuffer`。

五、综合编程题（共5题，每题4分，共20分）

1. 编写一个方法，接收一个整数数组，返回数组中的最大值。

参考代码：

```
public static int findMax(int[] arr) {  
    int max = arr[0];  
    for (int i = 1; i < arr.length; i++) {  
        if (arr[i] > max) {  
            max = arr[i];  
        }  
    }  
    return max;  
}
```

2. 编写代码，使用冒泡排序对数组 {5, 3, 8, 1, 9} 进行升序排序。

参考代码：

```
public static void bubbleSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = 0; j < arr.length - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
// 测试
public static void main(String[] args) {
    int[] arr = {5, 3, 8, 1, 9};
    bubbleSort(arr);
    for (int n : arr) System.out.print(n + " ");
    // 输出: 1 3 5 8 9
}
```

3. 使用字节缓冲流编写文件拷贝方法，将 "source.txt" 拷贝到 "target.txt"。

参考代码：

```
public static void copyFile() throws IOException {
    BufferedInputStream bis = new BufferedInputStream(
        new FileInputStream("source.txt"));
    BufferedOutputStream bos = new BufferedOutputStream(
        new FileOutputStream("target.txt"));
    byte[] buf = new byte[1024];
    int len;
    while ((len = bis.read(buf)) != -1) {
        bos.write(buf, 0, len);
    }
    bis.close();
    bos.close();
}
```

4. 设计一个多线程售票程序：3个窗口共同出售 100 张票，要求使用同步代码块保证线程安全。

参考代码：

```

class Ticket implements Runnable {
    private int tickets = 100;

    @Override
    public void run() {
        while (true) {
            synchronized (this) {
                if (tickets > 0) {
                    System.out.println(Thread.currentThread().getName()
                        + "卖出第" + tickets-- + "张票");
                } else {
                    break;
                }
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Ticket ticket = new Ticket();
        new Thread(ticket, "窗口1").start();
        new Thread(ticket, "窗口2").start();
        new Thread(ticket, "窗口3").start();
    }
}

```

5. 编写一个简单的 GUI 程序：创建一个窗口，包含一个按钮和一个标签，点击按钮时标签显示"Hello World"。

参考代码：

```
import javax.swing.*;
import java.awt.*;

public class MyFrame extends JFrame {
    public MyFrame() {
        setTitle("示例程序");
        setSize(400, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());

        JLabel label = new JLabel("等待点击");
        JButton btn = new JButton("点击我");

        btn.addActionListener(e ->
            label.setText("Hello World")
        );

        add(label);
        add(btn);
        setVisible(true);
    }

    public static void main(String[] args) {
        new MyFrame();
    }
}
```



第三部分 编程题常考题型模板

1. 数组类题型（冒泡排序、找最值）

冒泡排序模板：

```

public static void bubbleSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = 0; j < arr.length - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

找最大值：

```

public static int findMax(int[] arr) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}

```

2. 管理系统类题型（学生/图书/员工）

通用步骤：

1. 定义实体类（私有属性 + 构造方法 + getter/setter + toString）
2. 定义管理类（集合存储数据 + 增删改查方法）
3. 主类测试，添加数据并调用方法

3. 文件读写类题型

文本文件写入：

```

BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));
bw.write("内容");
bw.newLine();
bw.close();

```

文本文件读取：

```
BufferedReader br = new BufferedReader(new FileReader("input.txt"));
String line;
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
br.close();
```

4. 多线程售票题型（同步代码块）

 附录：易错点速记清单（考前必看！）

编号	易错点	正确理解
1	String 是基本类型	✗ String 是引用类型
2	float f = 1.1;	✗ 必须加 f/F： float f = 1.1f;
3	数组长度是方法 length()	✗ 数组长度是属性 length（无括号）
4	字符串比较用 ==	✗ 内容比较必须用 equals()
5	启动线程调用 run()	✗ 必须调用 start()
6	静态方法可以直接访问非静态成员	✗ 不能直接访问
7	finally 一定执行	✗ System.exit(0) 时不会执行
8	子类可以继承构造方法	✗ 构造方法不能继承，只能调用
9	ArrayList 增删快	✗ ArrayList 查询快、增删慢
10	HashSet 有序	✗ HashSet 无序，LinkedHashSet 有序
11	二维数组 int[][] a = new int[][3];	✗ 必须指定第一维长度
12	Integer 超出 127 仍用 == 比较	✗ 超出缓存池范围，== 比较地址
13	接口中所有方法都是抽象的	✗ JDK8+ 可有 default/static 方法

编号	易错点	正确理解
14	抽象类不能有构造方法	✗ 抽象类可以有构造方法（供子类调用）
15	throws 是在方法内抛出异常	✗ throws 在方法声明处，throw 在方法内
16	子类可以继承多个父类	✗ Java 是单继承，一个子类只能有一个直接父类
17	int 默认值是 null	✗ int 默认值是 0（引用类型默认才是 null）
18	线程调用 start() 后立即运行	✗ start() 使线程进入就绪状态，等待 CPU 调度
19	HashMap 是线程安全的	✗ HashMap 非线程安全，多线程用 ConcurrentHashMap
20	File 类可以读写文件内容	✗ File 只负责路径表示，不读写内容

 **最后提示：**本资料覆盖 Java 期末考试 **全部核心知识点**，面向对象（模块二）为重中之重，建议结合手写代码加深理解。

 **预祝考试顺利，成功上岸！**  